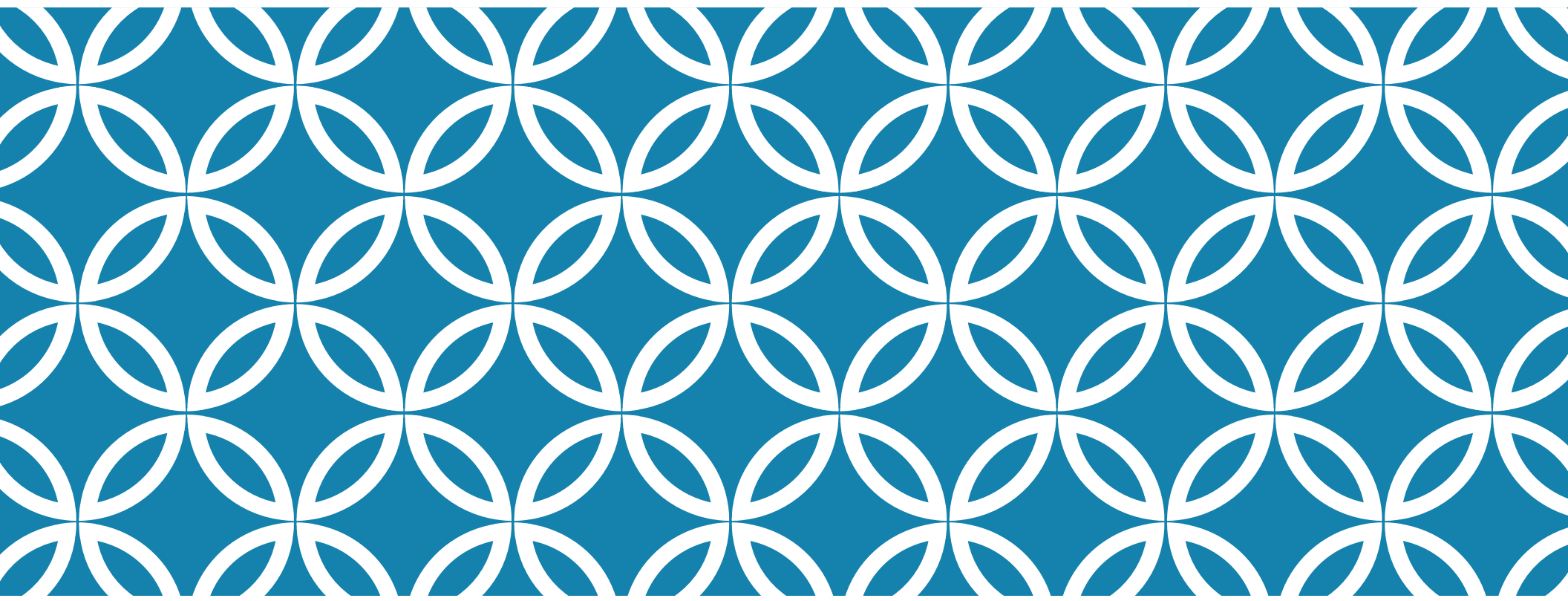




ARTIFICIAL INTELLIGENCE



REVISION

simple reflex agents

reflex agents with state

goal-based agents

utility-based agents

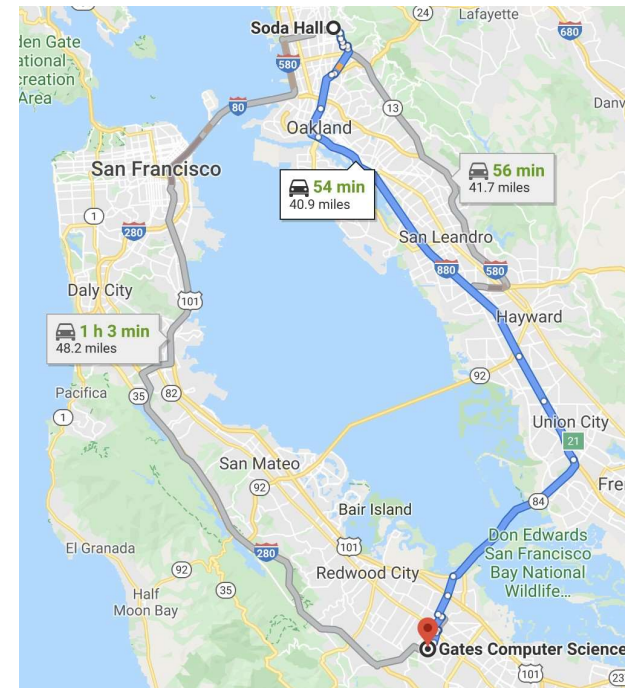
All of these 4 types can be turned into learning agents

PROBLEM SOLVING

Problem Solving Agents -> Atomic Representation

Assumption: Solution of a problem is a fixed sequence of actions and this sequence DOES NOT depend upon future percepts

APPLICATION: ROUTE FINDING



Objective: shortest? fastest? most scenic?

Actions: go straight, turn left, turn right

APPLICATION:ROBOT MOTION PLANNING

Objective: fastest path

Actions: acceleration and throttle



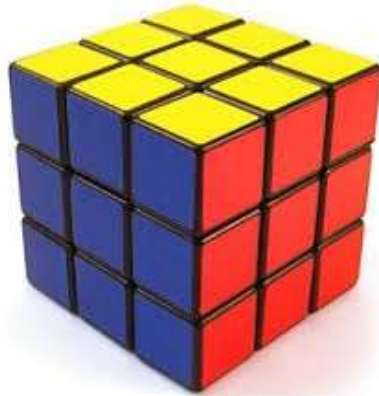
APPLICATION: ROBOT MOTION PLANNING



Objective: fastest? most energy efficient? safest? most expressive?

Actions: translate and rotate joints

APPLICATION: SOLVING PUZZLES



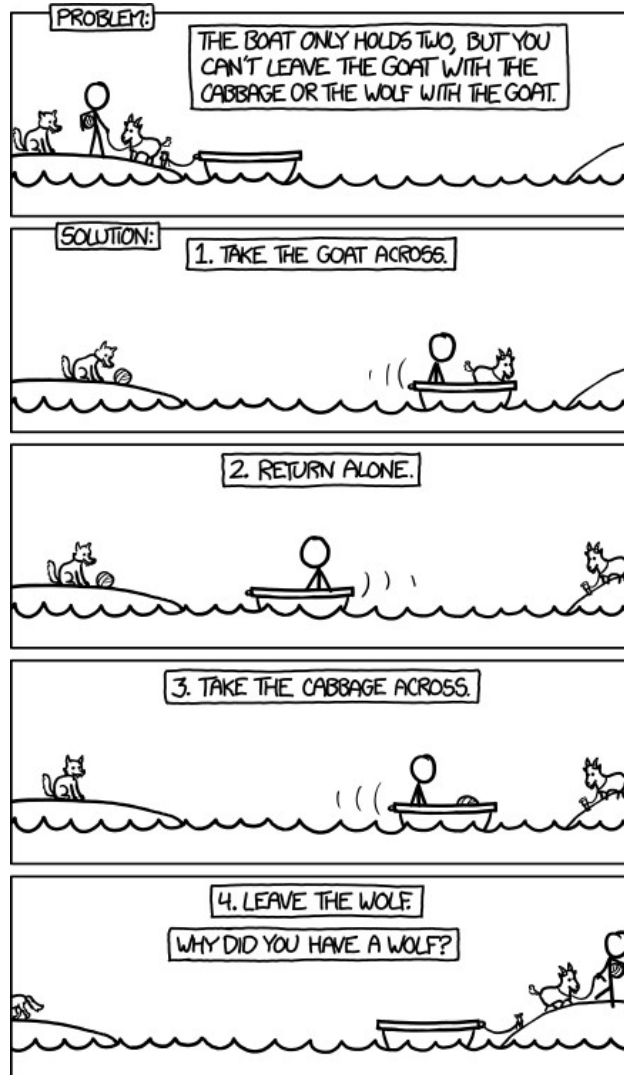
Objective: reach a certain configuration

Actions: move pieces (e.g., Move12Down)

QUESTION

A **farmer** wants to get his **cabbage**, **goat**, and **wolf** across a river. He has a boat that only holds two. He cannot leave the cabbage and goat alone or the goat and wolf alone. How many river crossings does he need?

- 4
- 5
- 6
- 7
- no solution



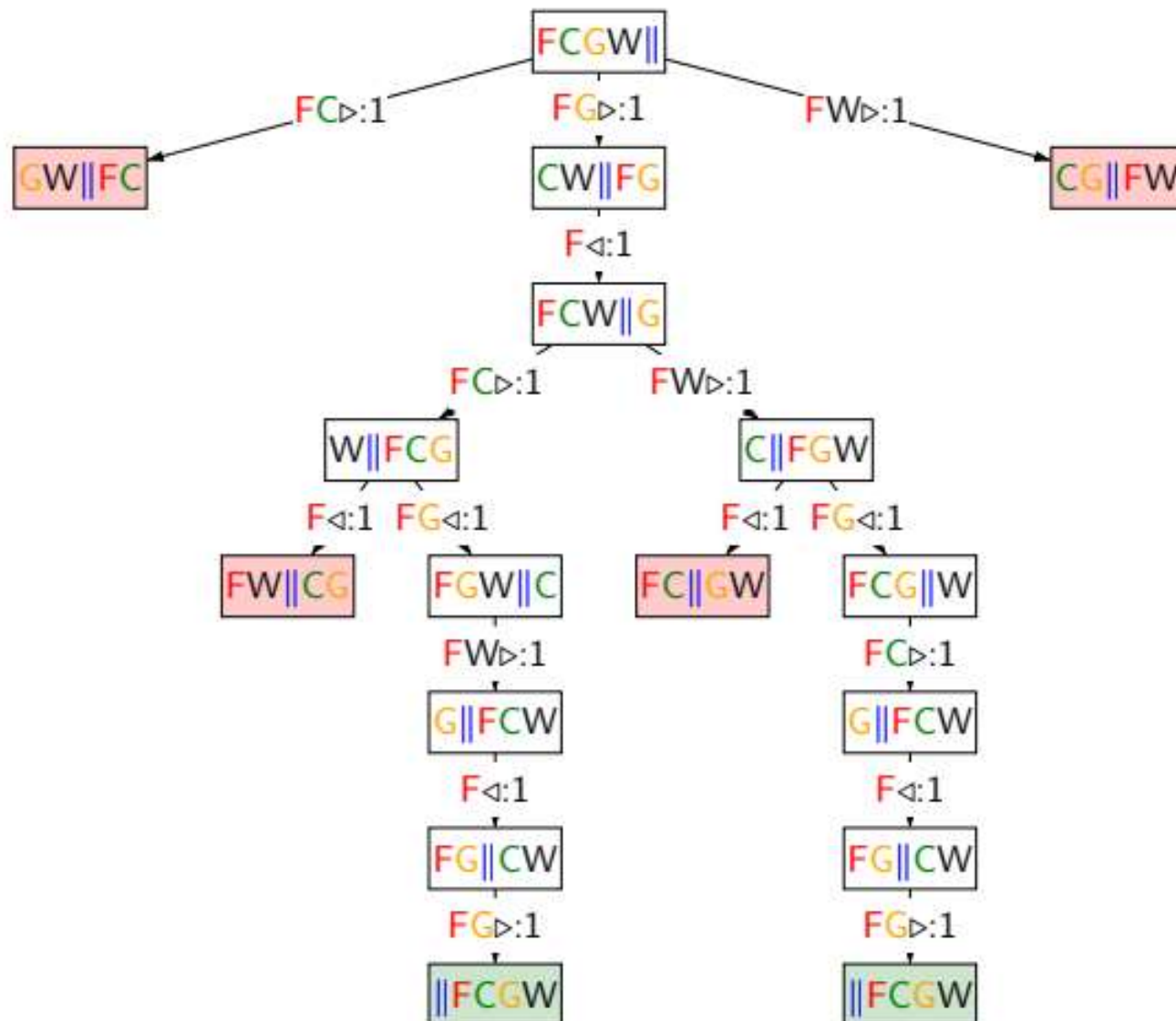


Farmer Cabbage Goat Wolf

Actions:

F▷	F◁
FC▷	FC◁
FG▷	FG◁
FW▷	FW◁

Approach: build a **search tree** ("what if?")



PROBLEM SOLVING

Problem Solving Agents -> Atomic Representation

Assumption: Solution of a problem is a fixed sequence of actions and this sequence DOES NOT depend upon future percepts

INFORMED VS UNINFORMED SEARCH

uninformed search algorithms—algorithms that are given no information about the problem other than its definition.

Informed search algorithms, on the other hand, can do quite well given some guidance on where to look for solutions

PROBLEM SOLVING AGENT

- 1) Goal Formulation
- 2) Problem Formulation
- 3) Searching Solution
- 4) Executing Solution

1- GOAL FORMULATION

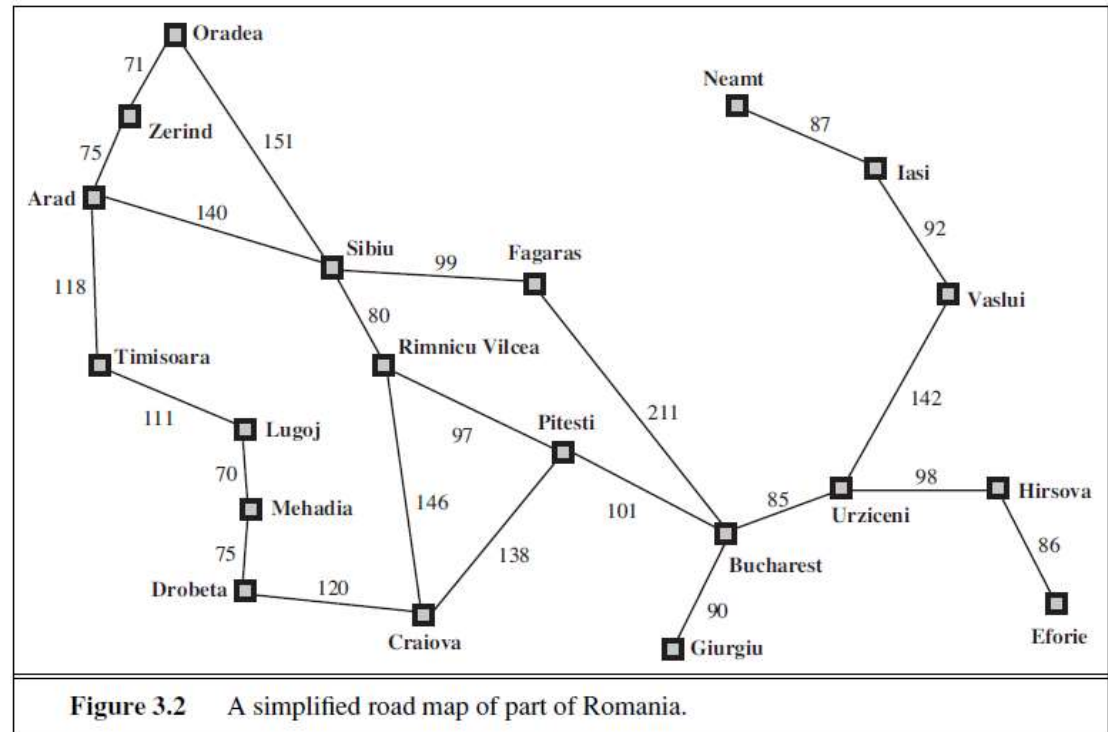
Goal formulation, based on the **current situation/state** and the agent's **performance measure**, is the first step in problem solving.

Performance Measure: Visit as many cities as possible, spend as low as possible on fuel.

Current State: In Arad



Possible Goal: In Bucharest



2- PROBLEM FORMULATION

Problem formulation is the process of deciding what actions and states to consider, given a goal.

The agent will consider actions at the level of driving from one major town to another. Each state therefore corresponds to being in a particular town.

Go Left, Go Right, Go forward, Go reverse. If these 4 actions are considered agent will never go out of the parking lot let alone reaching Bucharest.

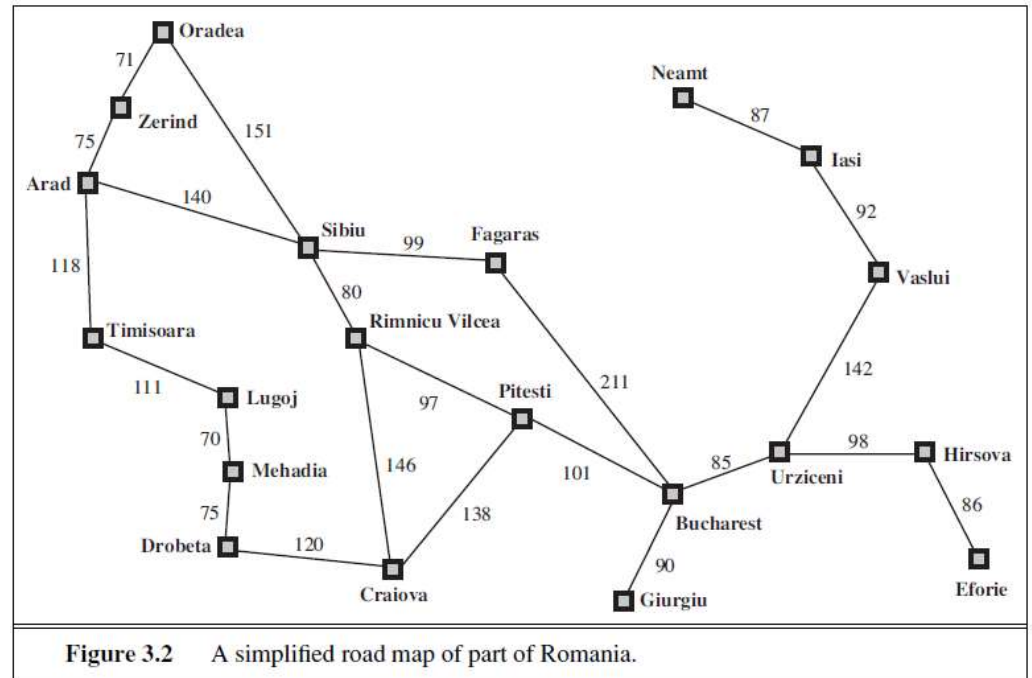
Possible Action: Go Cityname

STATE SPACE GRAPH

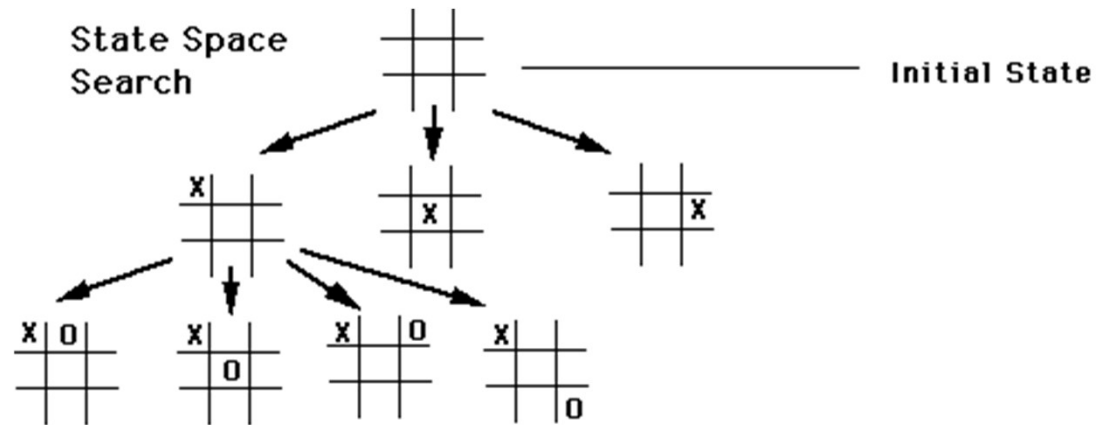
On the right is the state space graph of our problem

State Space - the set of all states reachable from the initial state by any sequence of actions.

A **path** in the state space is a sequence of states connected by a sequence of actions.

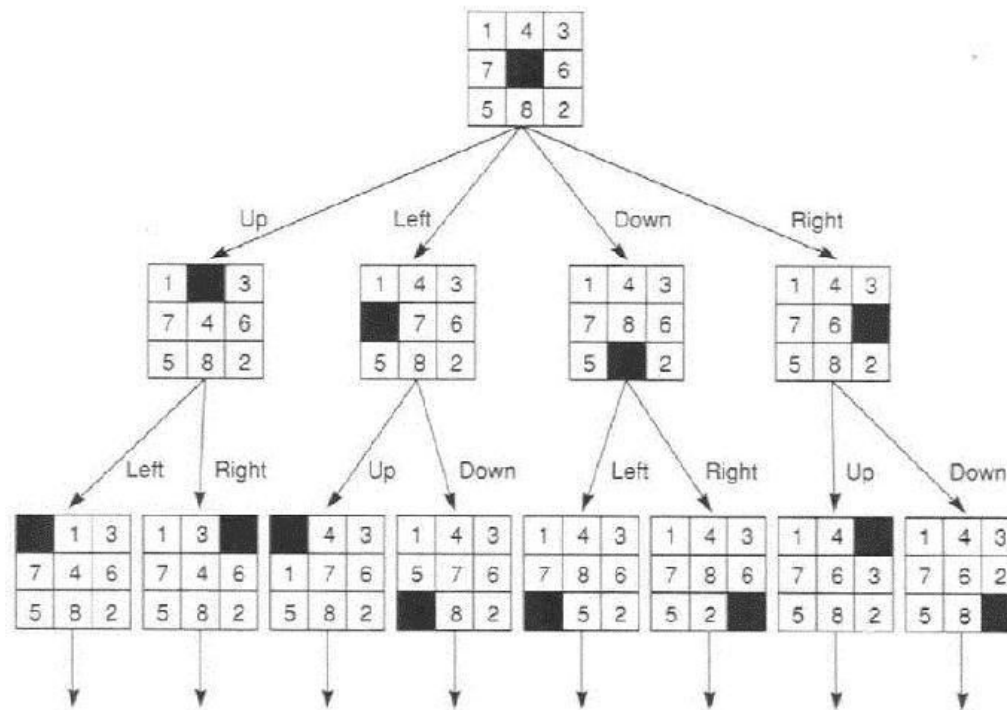


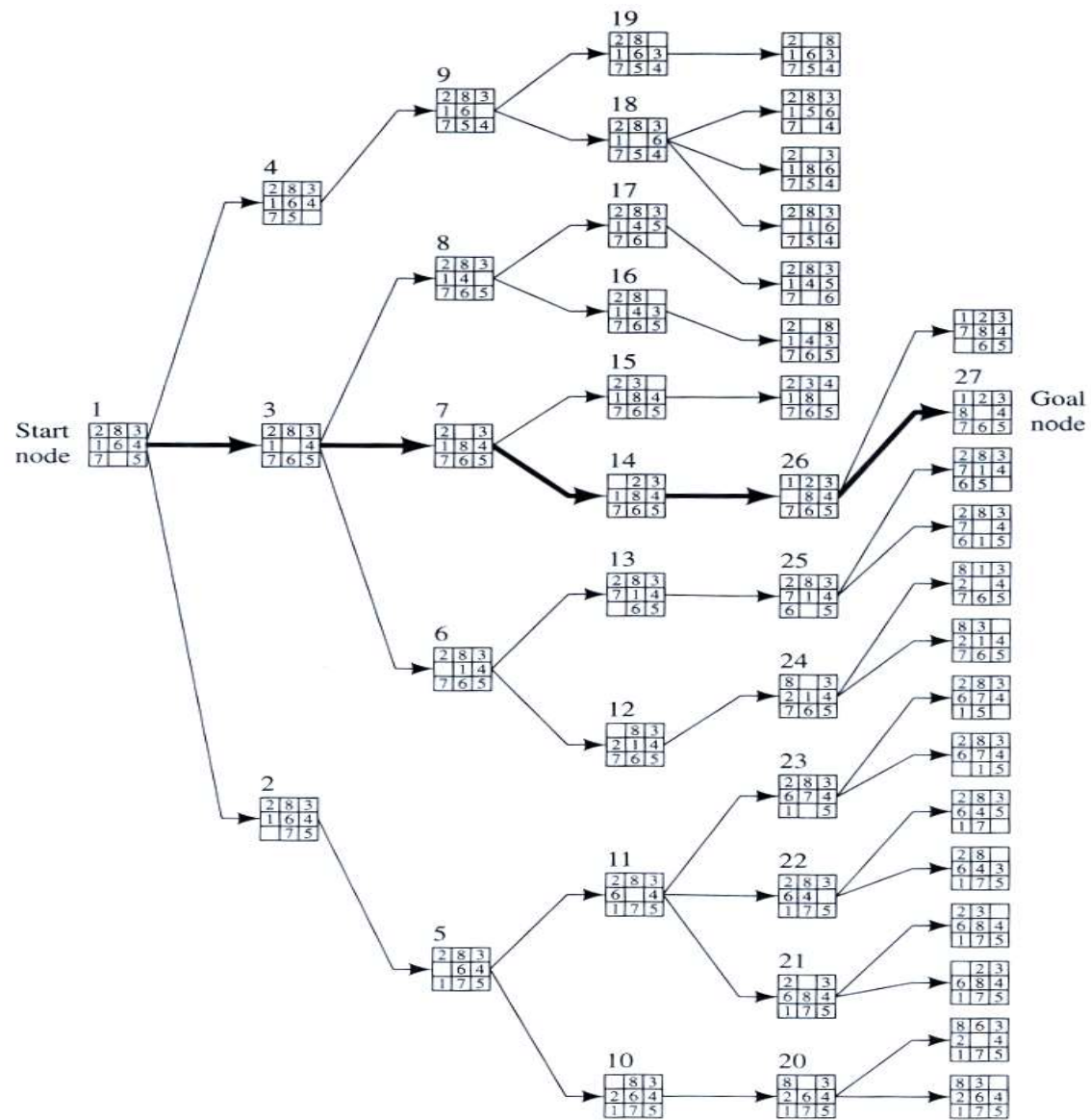
STATE SPACE GRAPH



How would a state space of chess look like?

TOY PROBLEM: 8 PUZZLE

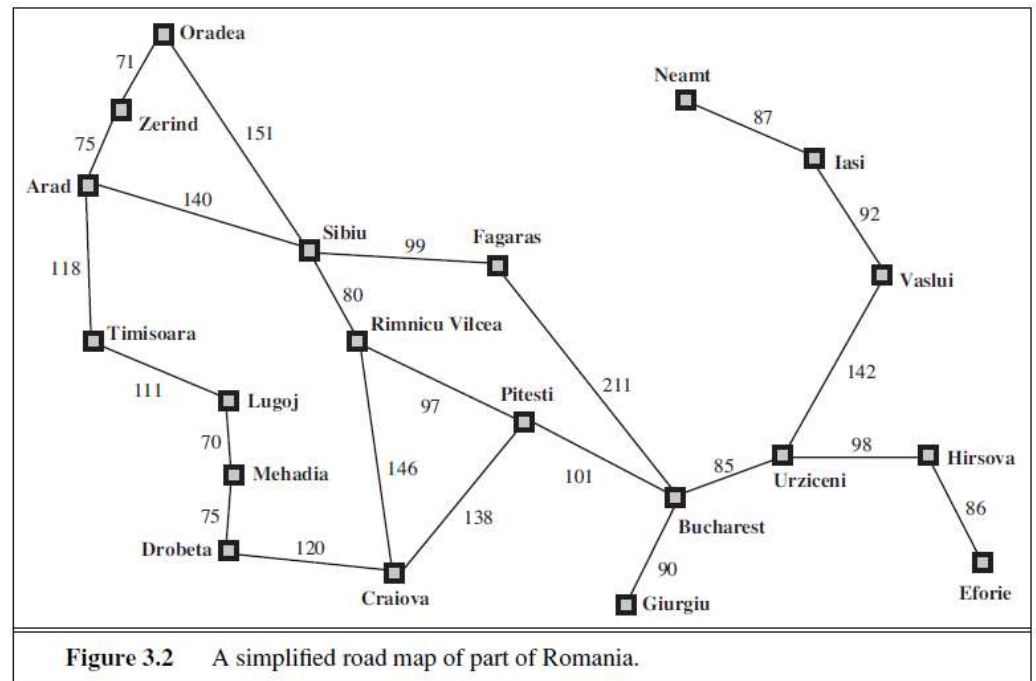




PROBLEM DEFINITION

A problem consists of five components,

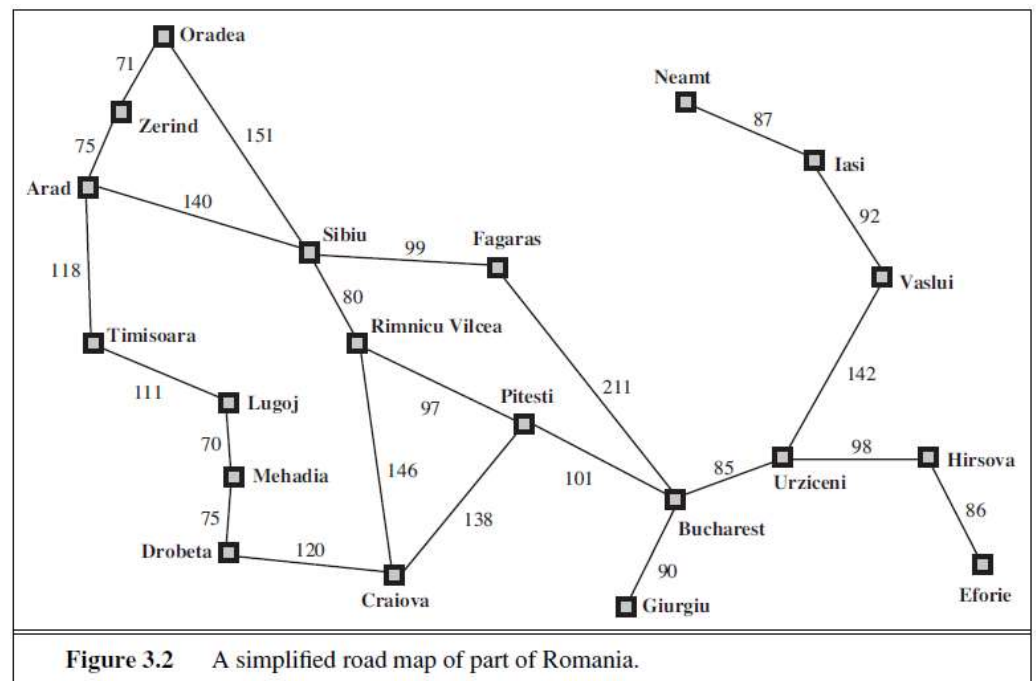
- Initial State
- Possible Actions
- Transition Model
- Goal Test
- Path Cost



PROBLEM DEFINITION

A problem consists of five components,

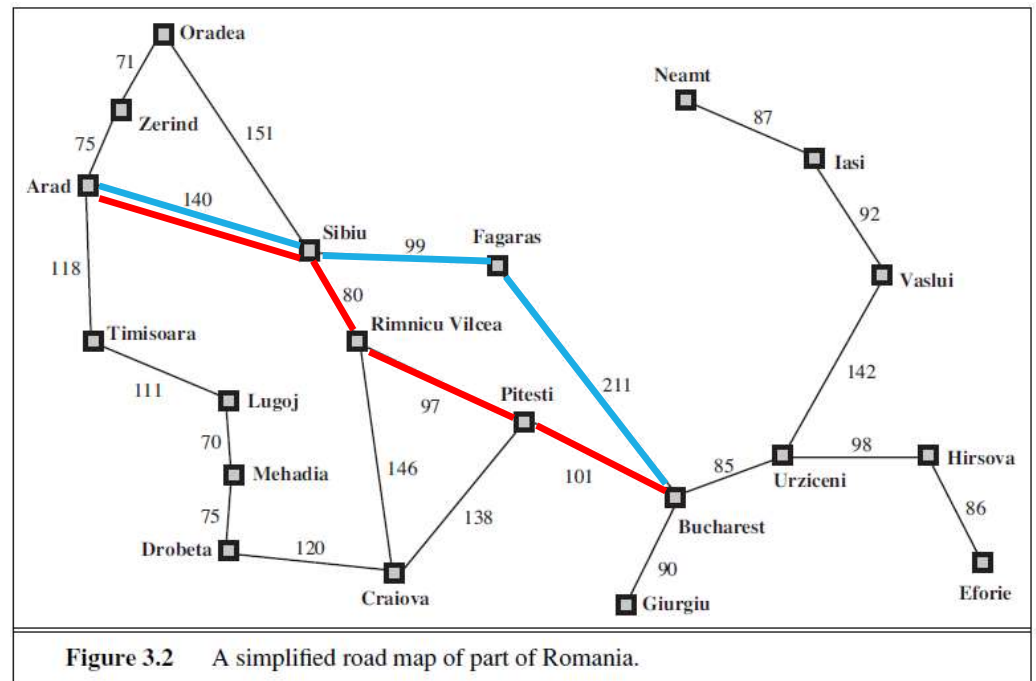
- Initial State : in (Arad)
- Possible Actions (given a state): e.g.,
- from Arad possible actions are $\{Go(Sibiu), Go(Timisoara), Go(Zerind)\}$.
- Transition Model: specifies the relationship between a state, a possible action and the resulting successor state e.g.,
 $RESULT(In(Arad), Go(Zerind)) = In(Zerind)$



PROBLEM DEFINITION

A problem consists of five components,

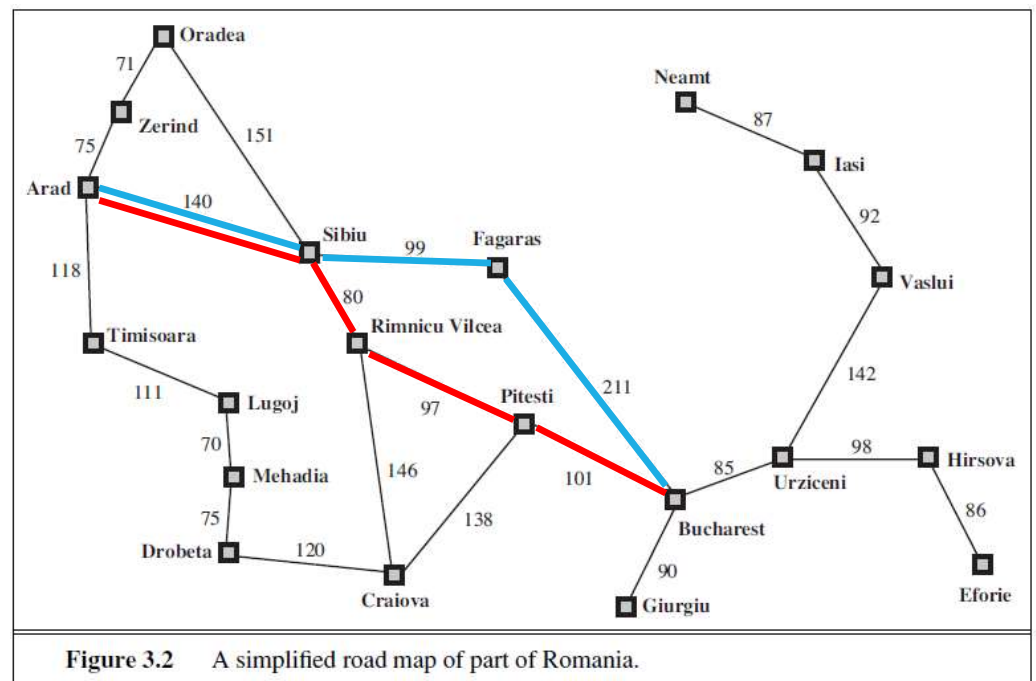
- Initial State
- Possible Actions
- Transition Model
- **Goal Test** (whether a given state is a goal state) e.g. $\{In(Bucharest)\}$.
- **Path Cost** (based upon agent's performance measure). Two paths shown on the right. **Step cost** (the cost of a single action within a path)



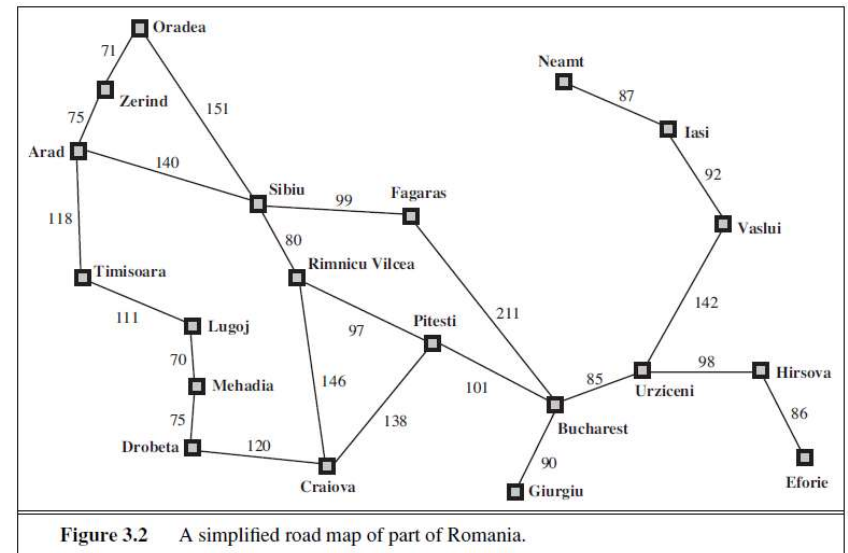
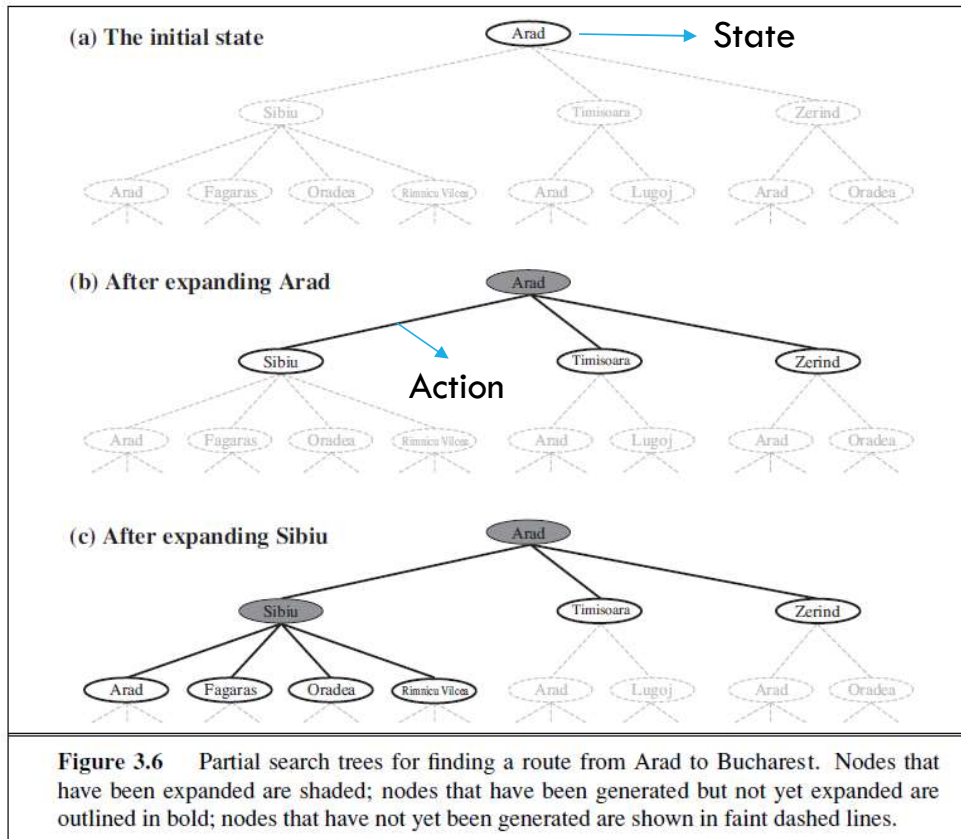
SOLUTION DEFINITION

A **solution** to a problem is an action sequence that leads from the initial state to a goal state.

Solution quality is measured by the path cost function, and an **optimal solution** has the lowest path cost among all solutions.

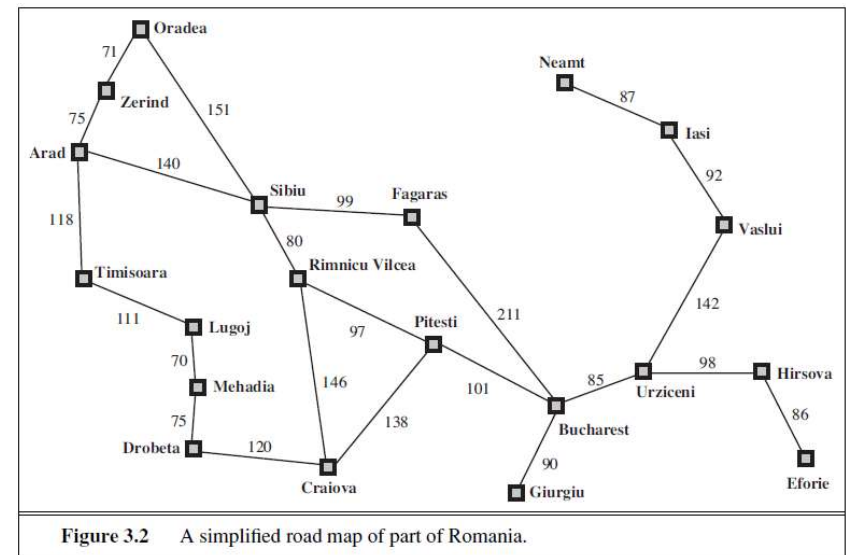
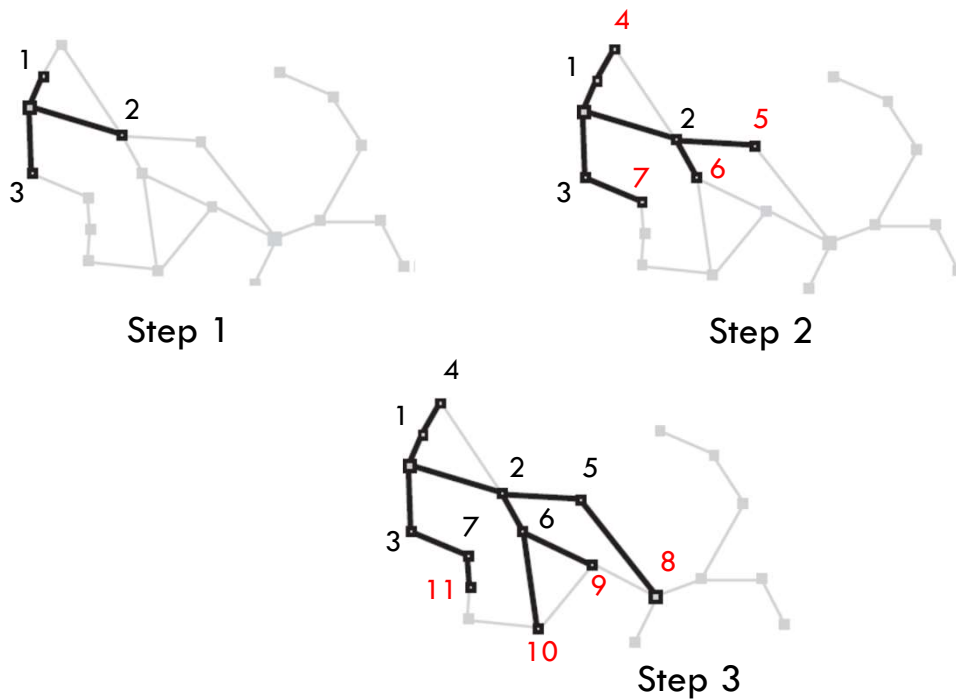


TREE SEARCH



GRAPH SEARCH

Use an **explore set** / **closed list** / **frontier** to remember the states already visited



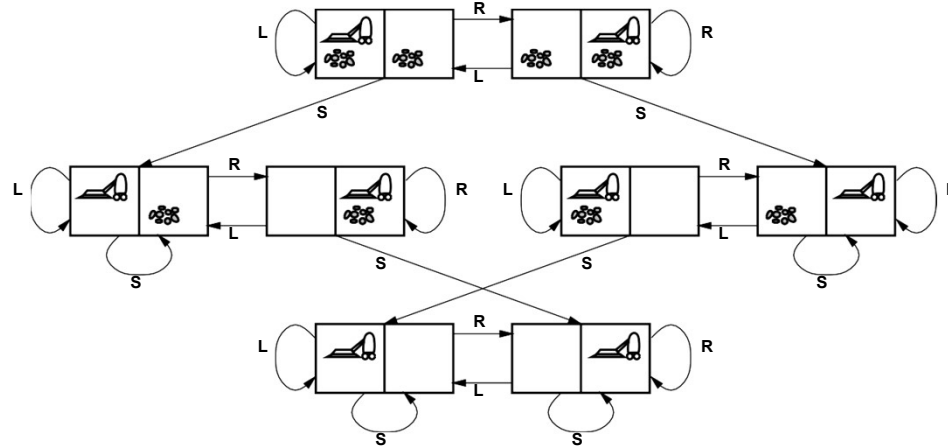
INFRASTRUCTURE FOR SEARCH ALGORITHMS

- n .STATE: the state in the state space to which the node corresponds;
- n .PARENT: the node in the search tree that generated this node;
- n .ACTION: the action that was applied to the parent to generate the node;
- n .PATH-COST: the cost, traditionally denoted by $g(n)$, of the path from the initial state to the node, as indicated by the parent pointers.

GOAL TEST

Frontier / expanded nodes / unexpanded nodes are stored in the form of a queue (can be FIFO, LIFO (stack) or priority queue)

EXAMPLE: VACUUM WORLD STATE SPACE GRAPH



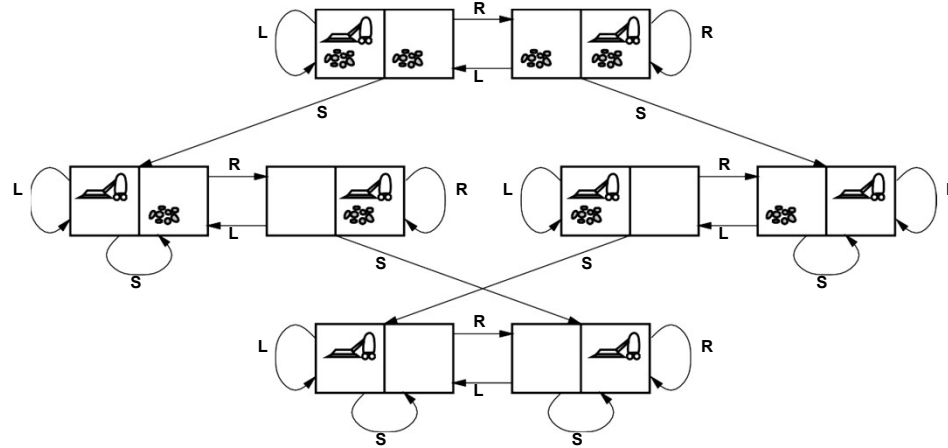
states??

actions??

goal test??

path cost??

EXAMPLE: VACUUM WORLD STATE SPACE GRAPH



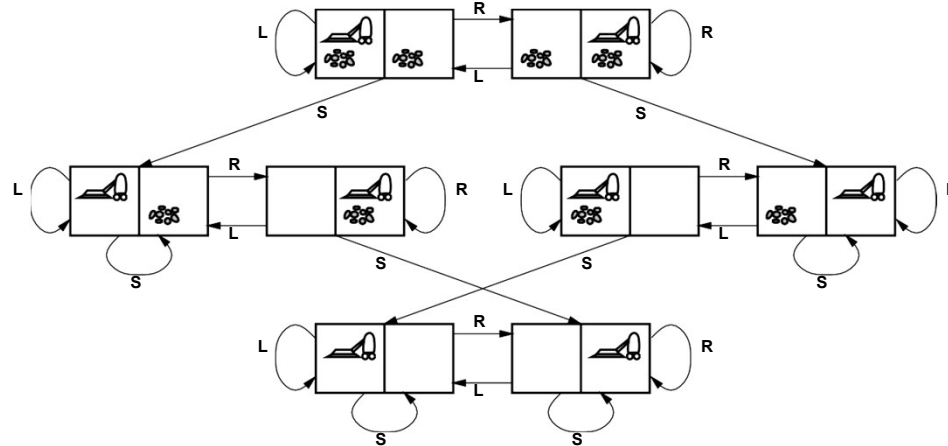
states??: integer dirt and robot locations (ignore dirt amounts etc.)

actions??

goal test??

path cost??

EXAMPLE: VACUUM WORLD STATE SPACE GRAPH



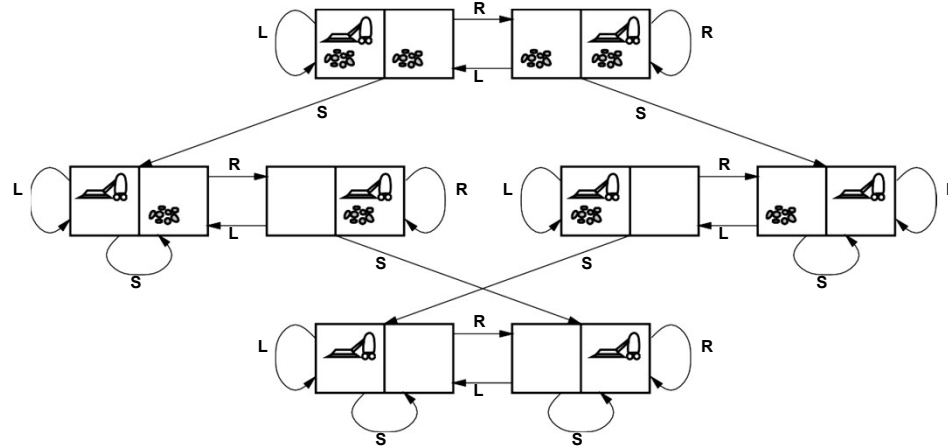
states??: integer dirt and robot locations (ignore dirt amounts etc.)

actions??: *Left, Right, Suck, NoOp*

goal test??

path cost??

EXAMPLE: VACUUM WORLD STATE SPACE GRAPH



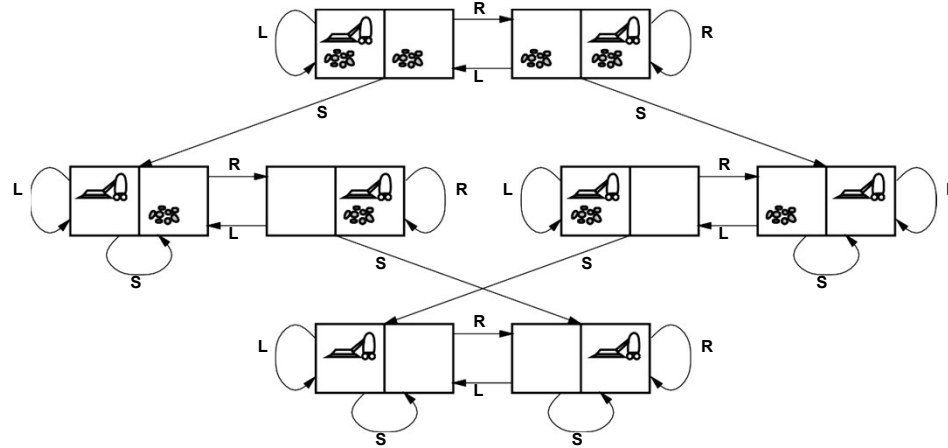
states??: integer dirt and robot locations (ignore dirt **amounts** etc.)

actions??: *Left, Right, Suck, NoOp*

goal test??: no dirt

path cost??

EXAMPLE: VACUUM WORLD STATE SPACE GRAPH



states??: integer dirt and robot locations (ignore dirt amounts etc.)

actions??: *Left, Right, Suck, NoOp*

goal test??: nodirt

path cost??: 1 per action (0 for *NoOp*)

EXAMPLE: THE 8-PUZZLE

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

states??

actions??

goal test??

path

cost??

EXAMPLE: THE 8-PUZZLE

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

states??: integer locations of tiles (ignore intermediate positions)

actions??

goal test??

path cost??

EXAMPLE: THE 8-PUZZLE

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

states??: integer locations of tiles (ignore intermediate positions)

actions??: move blank left, right, up, down (ignore unjamming etc.)

goal test??

path cost??

EXAMPLE: THE 8-PUZZLE

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

states??: integer locations of tiles (ignore intermediate positions)

actions??: move blank left, right, up, down (ignore unjamming etc.)

goal test??: = goal state(given)

path cost??

EXAMPLE: THE 8-PUZZLE

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

states??: integer locations of tiles (ignore intermediate positions)

actions??: move blank left, right, up, down (ignore unjamming etc.)

goal test??: = goal state(given)

path cost??: 1 permov

MEASURING PROBLEM-SOLVING PERFORMANCE

Completeness: Is the algorithm guaranteed to find a solution when there is one?

Optimality: Does the strategy find the optimal solution.

Time complexity: How long does it take to find a solution?

Space complexity: How much memory is needed to perform the search?

MEASURING PROBLEM-SOLVING PERFORMANCE

Complexity in a tree or a graph is measured by:

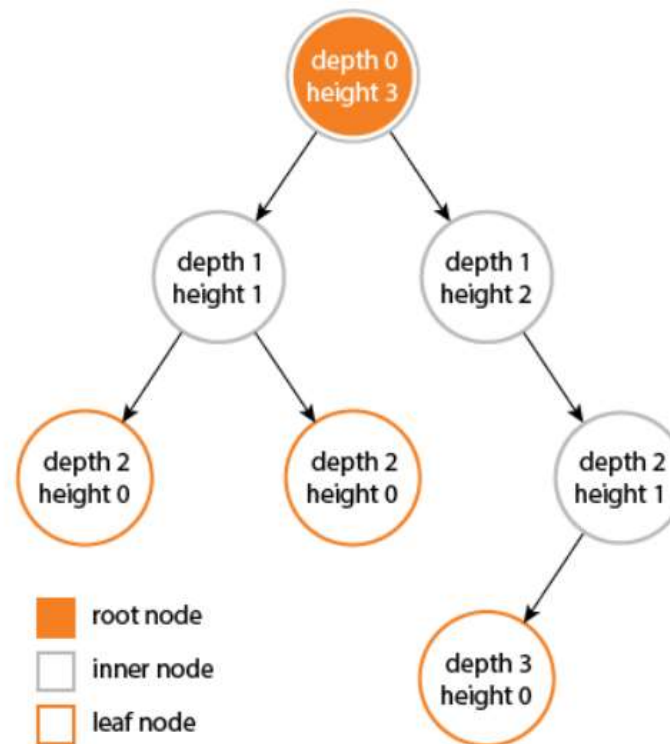
- 1) b , the **branching factor** or maximum number of children of any node
- 2) d , the **depth** of the goal node (i.e., the number of steps along the path from the root)
- 3) m , the maximum possible depth of entire tree.

Time is often measured in terms of the number of nodes generated during the search.

Space in terms of the maximum number of nodes stored in memory.

MEASURING PROBLEM-SOLVING PERFORMANCE

A binary tree has a branching factor



COMPLEXITY

Branching Factor, $b=3$ (Max num of children for any node is 3)

Depth, $d = 3$ (Bucharest is 3 steps away from Arad)

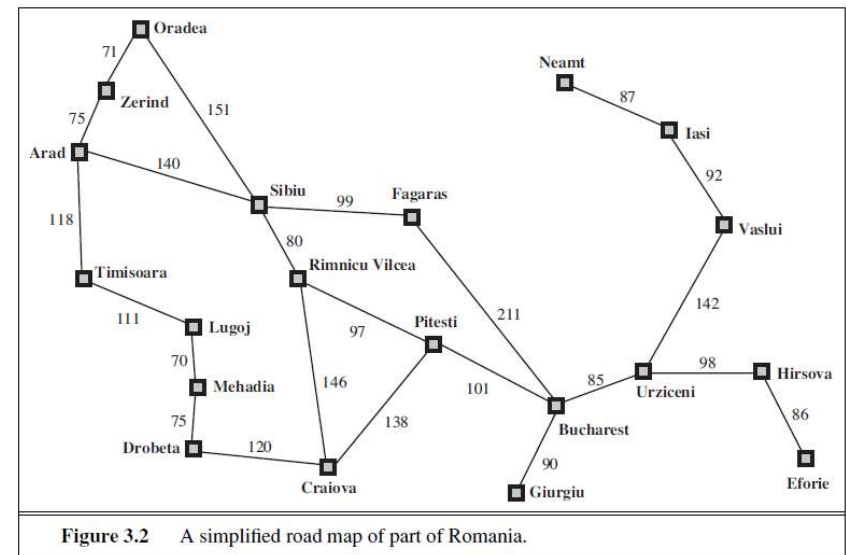
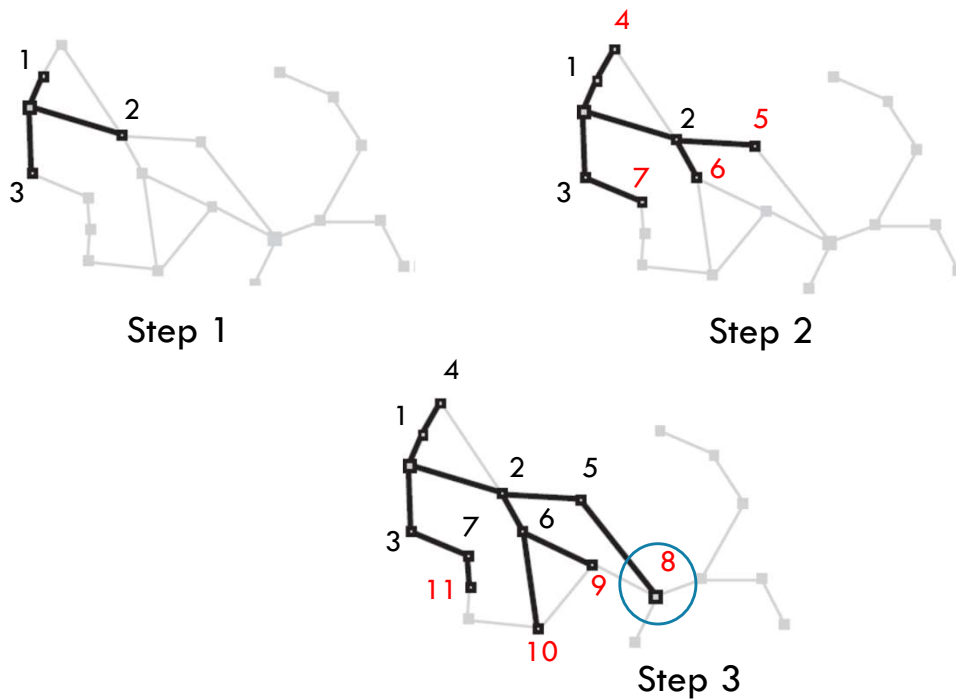


Figure 3.2 A simplified road map of part of Romania.